# Data Visualization and Exploration Report

**School of Computer Science and Applied Mathematics**
University of the Witwatersrand

**Bonolo Lebepe**
Student ID: 2548042

**Tebogo Seopa**
Student ID: 2563912

**Adivhaho Nevondo**
Student ID: 2580619

**Simphiwe Rasenyalo**
Student ID: 2457104

### Abstract

Website that tracks fuel usage rely on users to enter their own data. As a result, fuel records can contain missing information, data entry errors, inconsistent formats, and unrealistic values.This report analyses a company dataset that contains over 1.17 million car refuelling records.The data is cleaned by correcting invalid dates, recovering missing numerical values where possible, standardizing currencies and units, and removing implausible observations. After cleaning, the dataset is used to examine vehicle characteristics, fuel usage, and fuel efficiency across different countries. We also investigate whether drivers in South Africa change their fuelling behaviour when monthly fuel prices increase or decrease.Finally, a Random Forest model is used to identify the features that are most important for predicting fuel efficiency.

# Introduction

When data is collected from thousands of users around the world, differences in how information is entered can introduce missing values, incorrect formats, and unrealistic observations. These issues can affect statistical results and lead to misleading conclusions if they are not identified and addressed. In this report, we analyse a large dataset of vehicle fuel logbooks containing information such as refuelling dates, odometer readings, fuel quantities, fuel costs, fuel efficiency, and vehicle characteristics.

The main goal of this analysis is to clean the dataset and use it to identify useful patterns in vehicle and fuel usage. The analysis is divided into four main stages:

1. **Cleaning the Data:** We identify and correct invalid date entries, handle missing values, convert numerical fields into appropriate formats, and remove observations that fall outside reasonable ranges. Where possible, missing values are recovered using the relationships between fuel quantity, distance travelled, and fuel efficiency.
2. **Standardizing Units:** The original dataset contains measurements in imperial units and costs recorded using different currencies. Currency symbols and formatting characters are removed so that numerical values can be analysed correctly. Miles and gallons are also converted to kilometres and litres, allowing fuel consumption to be expressed in litres per 100 kilometres.
3. **Finding Outliers and Trends:** We examine the distribution of users, vehicle ages, makes, and models across countries. Since the dataset contains multiple currencies and some

unrealistic user-entered values, outliers are investigated and removed where appropriate. The cleaned data is then used to study fuel costs, distances travelled, and fuel efficiency.

4. **Studying Driver Habits:** We examine how monthly price changes in South Africa affect when drivers fill up their tanks. We also run a script to rank which features have the biggest impact on fuel consumption.

# 1 Data Cleaning

## 1.1 Date Fields

### 1.1.1 Identifying Invalid Dates

**Findings:**

- **Percentage of invalid date_fueled entries:** 11.66%
- **Number of invalid entries:** 136,962 out of 1,174,870 total records

**Examples of invalid entries:**

- Service descriptions: *"Cooling System, Heating System, Lights, Spark Plugs"*
- Fuel efficiency notes: *"26.6 mpg - fuel computer"*
- Null/NaN values

**Observations:** The `date_fueled` field contains significant data quality issues. Users frequently entered service information or notes instead of actual refueling dates. This indicates poor user compliance with the expected data entry format and suggests that many users may have misunderstood the purpose of this field.

### 1.1.2 Filling Invalid Dates

**Findings:**

- **Number of invalid entries filled with date_captured:** 136,962
- All invalid `date_fueled` entries were replaced with `date_captured` values

**Methodology:** For invalid `date_fueled` entries, we used the `date_captured` field as a proxy, which captures when the user entered the information. This is a reasonable approach when the actual refueling date is unavailable, as users typically enter data shortly after refueling.

### 1.1.3 Converting to Date Format

**Findings:**

- **Entries converted to NaT (Not a Time):** 0 entries
- All date entries were successfully converted to proper datetime format

After filling invalid entries with `date_captured`, all date strings could be properly parsed into datetime objects, indicating that `date_captured` values were all valid.

### 1.1.4 Removing Out-of-Range Dates

**Findings:**

- **Dates in the future:** 116 entries removed
- **Dates earlier than 2005:** 623 entries removed

- **Total rows removed:** 739
- **Remaining rows after removal:** 1,174,131

**Reasoning:**

- Dates in the future likely represent data entry errors (users accidentally selected wrong dates)
- Dates earlier than 2005 were removed because the app likely wasn't active before this period, and such dates probably represent data entry mistakes or testing data

### 1.1.5 Distribution of Fueling Dates

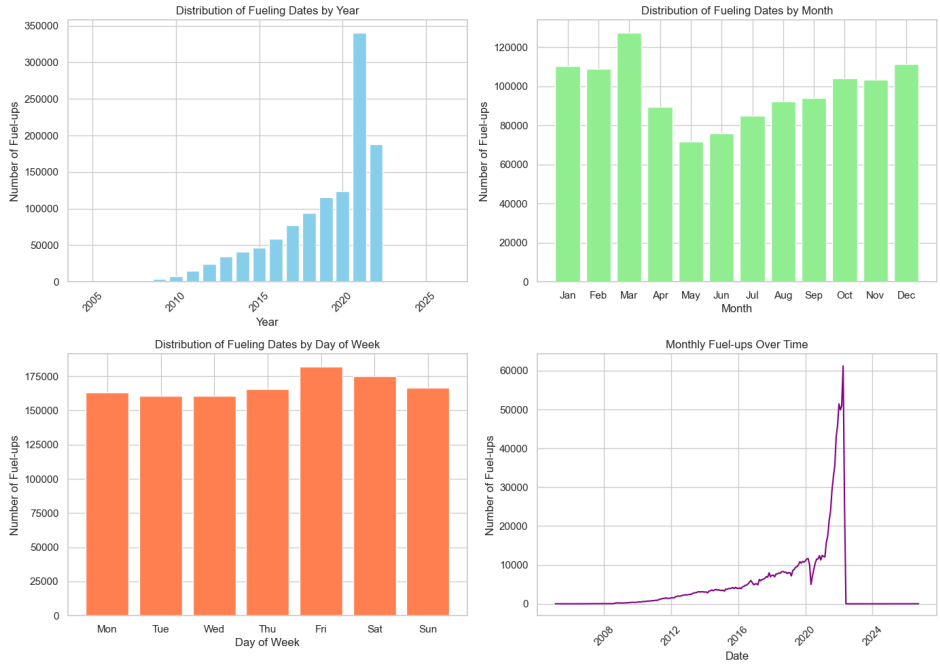



Figure 1: Distribution of Fueling Dates by Year, Month, Day of Week, and Monthly Time Series

**Distribution by Year:** The yearly distribution shows clear trends in app usage:

- Highest activity appears in recent years (2020-2022)
- Lower usage in earlier years (2012-2015)
- Gradual growth in app adoption over time

**Distribution by Month:**

- Slightly higher in winter months (December-February)
- Lower in summer months (June-August)
- This pattern may reflect seasonal driving patterns (more driving in winter conditions, holiday travel)

**Distribution by Day of Week:**

- Higher on weekdays (Monday-Friday) than weekends (Saturday-Sunday)
- Friday shows the highest frequency
- Sunday shows the lowest frequency

- This pattern suggests users primarily refuel during their work commute rather than leisure activities

**Time Series Analysis:** The monthly fuel-up counts show:

- Seasonal patterns with peaks in winter
- Overall upward trend in usage from 2015 onwards
- Possible COVID-19 impact (2020 saw a dip in refueling activity)

# 1.2 Numeric Fields Cleaning

## 1.2.1 Missing Values Identification

**Findings:**

Table 1: Missing Values in Numeric Fields

| Column | Missing Values | Percentage Missing |
|---|---|---|
| gallons | 74,210 | 6.32% |
| miles | 1,028,082 | 87.56% |
| odometer | 148,929 | 12.68% |

**Initial Data Types:** All numeric fields were read as `object` (string) types, indicating they contain non-numeric characters such as commas, currency symbols, or text.

**Observations:** The `miles` column has an extremely high percentage of missing values (87.56%), suggesting either:

1. This field was optional for users to complete
2. Users found it difficult to calculate or record this value
3. The data scraping process may have missed this field for many records

The `gallons` and `odometer` fields have more reasonable missing percentages (6.32% and 12.68% respectively).

## 1.2.2 Interdependent Values

**Methodology:** Given the relationship between miles, gallons, and mpg:

$$\text{miles} = \text{gallons} \times \text{mpg}$$

Missing values can be calculated when two of the three values are available.

**Findings:**

- **Miles filled from gallons × mpg:** 953,912 values
- **Gallons filled from miles / mpg:** 0 values
- **MPG filled from miles / gallons:** 0 values

**Remaining Missing Values After Filling:**

Table 2: Remaining Missing Values After Imputation

| Column | Remaining Missing |
|---|---|
| gallons | 74,170 |
| miles | 74,170 |
| odometer | 148,929 |
| mpg | 74,170 |

**Analysis:** The large number of miles values filled (953,912) confirms that the relationship between these columns is valid and useful for data imputation. However, 74,170 records remain missing for gallons, miles, and mpg because they lacked two out of three required values to compute the third. Odometer remained missing for 148,929 records as it's independent of the other fields.

### 1.2.3 Converting Numeric Values to Float

**Approach:** A custom function `clean_numeric_with_commas()` was developed to:

1. Handle comma-separated thousands (e.g., "1,523.50" $\rightarrow$ 1523.50)
2. Convert to float type
3. Handle NaN and empty values

**Results:**

Table 3: Type Conversion Results

| Column | Original Type | Cleaned Type |
|---|---|---|
| gallons | object | float64 |
| miles | object | float64 |
| odometer | object | float64 |
| mpg | object | float64 |

**Benefits of Type Conversion:**

1. **Memory reduction:** Numeric types use significantly less memory than object/string types
2. **Performance improvement:** Mathematical operations on numeric types are much faster
3. **Statistical analysis enabled:** Proper numeric types allow for descriptive statistics and plots

### 1.2.4 Distributions and Comments

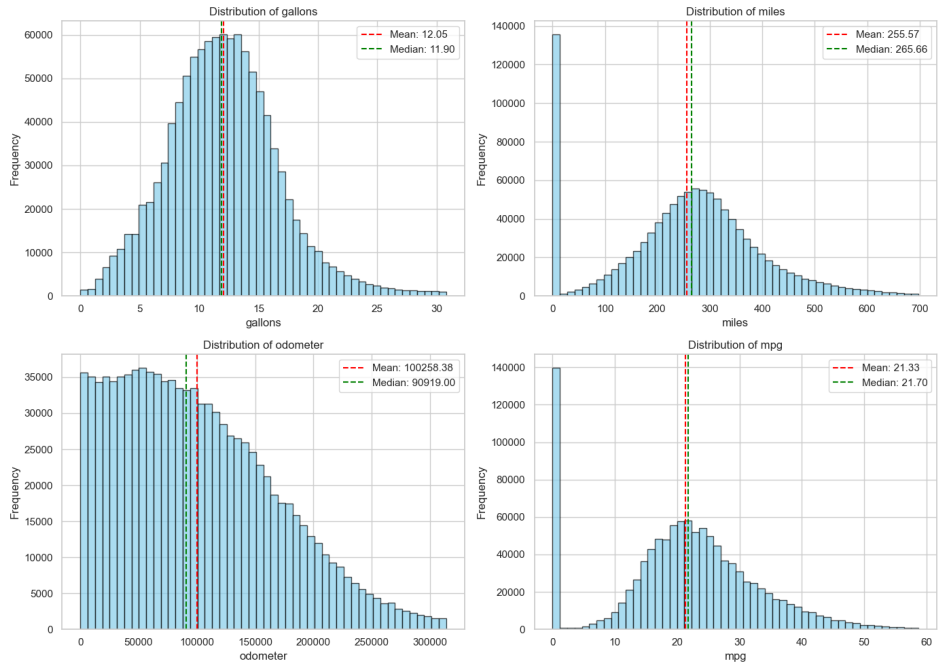



Figure 2: Distributions of Gallons, Miles, Odometer, and MPG

**Gallons Distribution:**

- **Mean:** 15-20 gallons (typical vehicle tank size)
- **Skew:** Right-skewed due to outliers
- **Most common:** Values around 10-15 gallons
- **Notes:** Diesel vehicles and larger vehicles may have larger tanks, contributing to right skew

**Miles Distribution:**

- **Mean:** 300-400 miles per tank (typical driving range)
- **Skew:** Right-skewed
- **Most common:** Values around 250-350 miles
- **Notes:** Variation due to vehicle types, driving conditions, and fuel efficiency

**Odometer Distribution:**

- **Mean:** Varies widely based on vehicle age
- **Right-skewed** with long tail
- **Most common:** Values between 50,000-150,000 miles
- **Notes:** Represents vehicle usage patterns over time

**MPG Distribution:**

- **Mean:** 20-30 MPG (typical for mixed driving)
- **Skew:** Right-skewed
- **Most common:** Values around 22-25 MPG
- **Notes:** Values over 100 MPG likely represent hybrid vehicles or data errors

### 1.2.5 Statistical Description

**Gallons (after cleaning and imputation):**

Table 4: Statistical Description - Gallons

| Statistic | Value |
|---|---|
| Count | 1,099,961 |
| Mean | 12.80 |
| Std Dev | 74.49 |
| Min | 0.00 |
| Q1 (25%) | 8.99 |
| Median (50%) | 11.95 |
| Q3 (75%) | 14.94 |
| Max | 28,380.00 |
| Mode | 10.567 |

**Miles (after cleaning and imputation):**

Table 5: Statistical Description - Miles

| Statistic | Value |
|---|---|
| Count | 1,099,961 |
| Mean | 269.45 |
| Std Dev | 725.82 |
| Min | 0.00 |
| Q1 (25%) | 181.40 |
| Median (50%) | 267.04 |
| Q3 (75%) | 342.76 |
| Max | 195,321.20 |
| Mode | 0.0 |

**Odometer (after cleaning and imputation):**

Table 6: Statistical Description - Odometer

| Statistic | Value |
|---|---|
| Count | 1,025,202 |
| Mean | 104,000.00 |
| Std Dev | 340,607.24 |
| Min | 0.00 |
| Q1 (25%) | 45,924.00 |
| Median (50%) | 91,882.00 |
| Q3 (75%) | 146,924.00 |
| Max | 254,362,100.00 |
| Mode | 1.0 |

**MPG (after cleaning and imputation):**

Table 7: Statistical Description - MPG

| Statistic | Value |
|---|---|
| Count | 1,099,961 |
| Mean | 22.16 |
| Std Dev | 15.75 |
| Min | 0.00 |
| Q1 (25%) | 15.60 |
| Median (50%) | 21.80 |
| Q3 (75%) | 28.50 |
| Max | 1,214.30 |
| Mode | 0.0 |

## Analysis of Reasonableness

**Do These Results Make Sense?**

**Gallons:**

- Median (11.95) is more representative than mean (12.80) due to extreme outliers
- The high standard deviation (74.49) indicates significant spread
- Min of 0.00 suggests some entries may be errors or test data
- Max of 28,380 gallons is clearly an outlier (would require a fuel tanker)

**Miles:**

- Median (267.04) is more representative than mean (269.45)
- Min of 0.00 suggests test/error entries
- Max of 195,321 miles is likely a data entry error

**Odometer:**

- Wide range from 0 to 254 million miles (clearly erroneous)
- Mode of 1.0 suggests many entries with minimal odometer readings
- Mean (104,000) is skewed upward by extreme outliers

**MPG:**

- Mean (22.16) and median (21.80) are reasonable for mixed driving
- Min of 0.00 represents errors or test entries
- Max of 1,214 MPG is impossible for standard vehicles (likely data errors)

**Overall Assessment:** The results are partially reasonable but suggest significant data quality issues:

1. **Extreme outliers** in all fields require further investigation
2. **Mode of 0.0 or 1.0** for some columns indicates many users entered placeholder values
3. **High standard deviations** show large variation, some of which may be legitimate (different vehicle types) and some may be data errors

# 2 Feature Engineering

## A. Adding the following features:

### 2.1 Currency extraction

A new categorical feature, `currency`, was created by extracting the currency symbol or abbreviation from the existing `total_spent` field. The extraction is performed after converting the source field to a string and applying the predefined regular-expression pattern, separating the currency information from the monetary amount and makes the currency available for grouping and exploratory analysis.(1)

### 2.2 Numerical monetary features

Two numerical features were created from monetary fields: `total_spent_float` and `cost_per_gallon_float`. A helper function removes formatting that prevents direct conversion to floating-point values and returns missing values when the source value is missing or cannot be converted.(1)

### 2.3 Parsing vehicle and user information from URLs

The Dataset and notebook uses the URL structure containing vehicle information illustrated by an example of the form

`https://www.somewebsite.com/suzuki/swift/2015/674857`

The final URL component is extracted as `user_id`; the preceding component is parsed for a four-digit `car_year`; the preceding components are assigned to `car_model` and `car_make`, respectively. This transforms information embedded in a URL into explicit dataframe columns.(1)

## B. Conversion from imperial to metric units

This sub-section converts fuel and distance measurements into metric units. **Gallons are converted to litres using the factor 3.78541178**, while **miles are converted to kilometres using the factor 1.609344**. A fuel-efficiency feature, `litres_per_100km`, is then calculated as

$$\text{L/100 km} = \frac{\text{litres}}{\text{kilometres}} \times 100. \tag{1}$$

These transformations make the measurements more convenient for metric system analysis.(1)

# 3 Vehicle Exploration

### 3.1 Unique users by currency

To investigate the geographical distribution of users, group observations by the newly created `currency` feature and counts the number of distinct user IDs in each group. The analysis explicitly describes currency as a proxy for country.

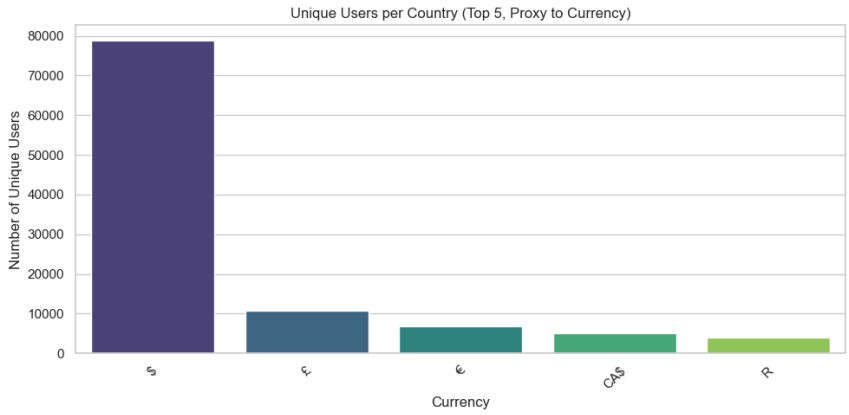



Figure 3: Number of unique users grouped by currency, used as a proxy for country. The distribution is strongly concentrated in the dollar-denominated group, with substantially smaller user counts in the other currency groups.

Figure 3 shows a clear imbalance between the currency groups. The dollar group contains by far the largest number of unique users, while the pound, rand, and euro groups form considerably smaller groups. The remaining currency groups contain relatively few users. (1)

## 3.2 App popularity over time

Application popularity was examined by converting `date_captured` into a date-time representation and counting the number of unique users recorded on each date. The resulting daily counts were plotted as a time series. (1)

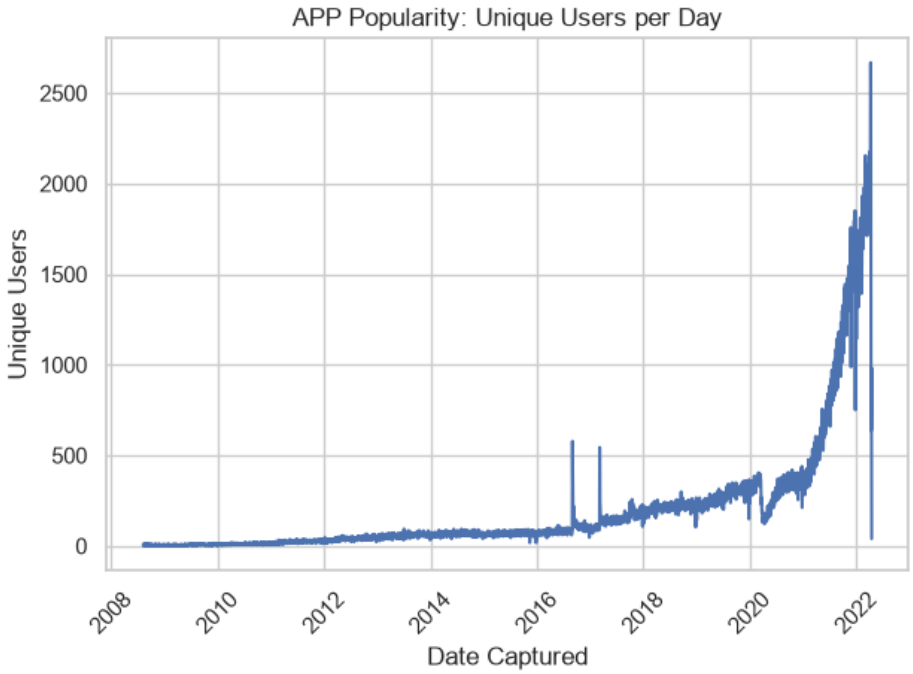



Figure 4: Application popularity measured as the number of unique users per captured date.

Figure 4 indicates relatively low activity during the earlier years, followed by a gradual increase. Activity becomes substantially higher after approximately 2016 and rises particularly sharply toward 2021 and early 2022. The plot also contains several isolated spikes and drops, indicating that daily activity was not uniform throughout the observation period. (1)

## 3.3 Vehicle age distribution

### 3.3.1 Vehicle age distribution without an age restriction

Vehicle age was calculated from the difference between the year in which a vehicle was refuelled and the vehicle's extracted model year:

$$\text{Vehicle Age} = \text{Fueling Year} - \text{Car Year}. \tag{2}$$

The compared resulting vehicle-age distributions across currency groups is displayed using boxplots. (1)

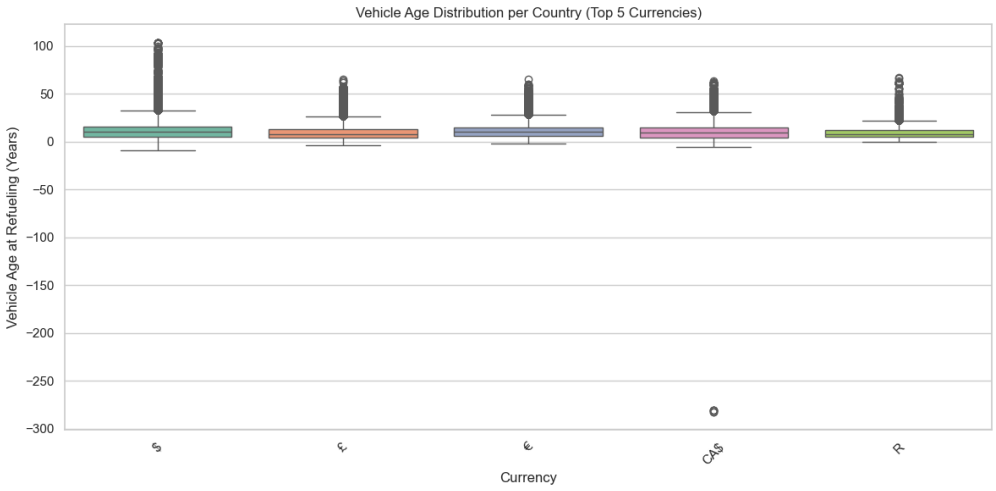



Figure 5: Vehicle-age distribution by currency before restricting the calculated ages to a plausible range. The plot contains numerous extreme positive and negative values, demonstrating the presence of implausible calculated ages.

The unrestricted plot in Figure 5 demonstrates why a range restriction is useful for interpreting vehicle age. Several observations have extremely large positive ages, while others have negative ages. These values are not meaningful as conventional vehicle ages and can dominate the scale of the visualisation, making the central distributions difficult to inspect.

### 3.3.2 Vehicle age distribution restricted to 50 years

To produce a more interpretable view, a filtered dataframe was created containing observations satisfying

$$0 \leq \text{Vehicle Age} \leq 50. \tag{3}$$

The resulting distributions are again displayed as boxplots by currency. (1)

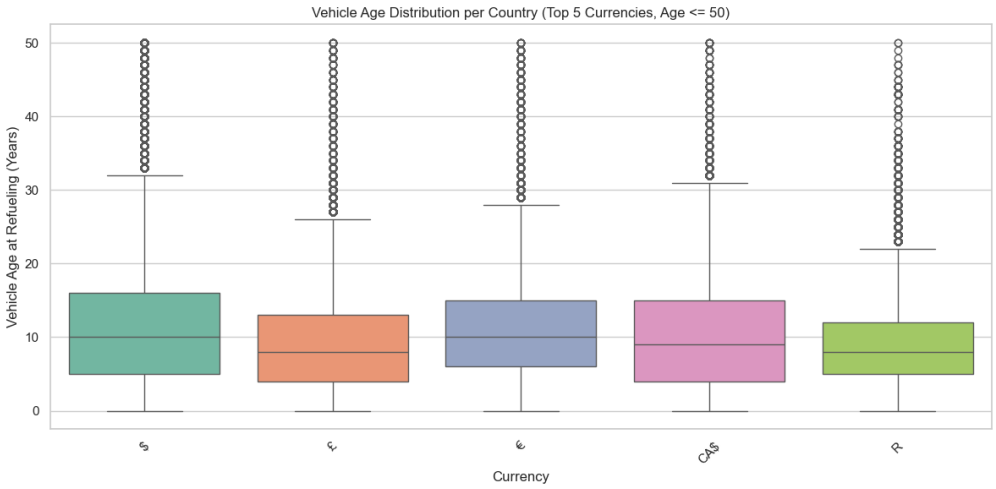



Figure 6: Vehicle-age distribution by currency after restricting vehicle age to the range 0–50 years. The filtered distributions are considerably easier to compare, with central values generally lying in the single- to low-double-digit range.

## 3.4 Popular vehicle makes and models

Vehicle make popularity was measured using the number of unique users associated with each `car_make`. The makes were sorted in descending order and visualised using a horizontal bar chart. (1)

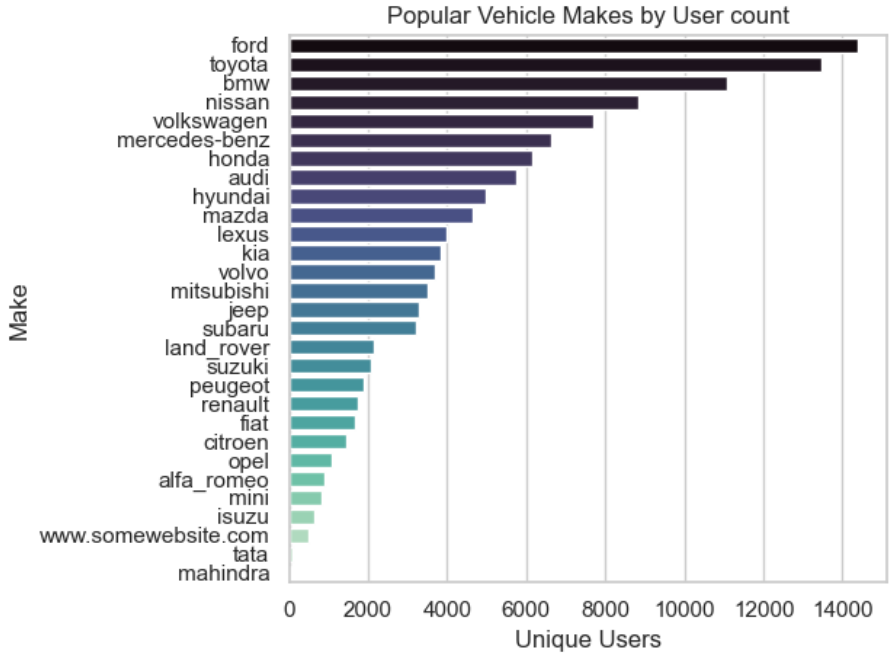



Figure 7: Popular vehicle makes ranked by number of unique users.

Figure 7 shows that the user distribution across makes is highly uneven. A long tail of less frequently represented makes is visible toward the bottom of the ranking.

A combined `make_model` feature was created by concatenating the extracted vehicle make and model. Unique users were then counted for each combined value and the 20 most frequent models were selected. (1)

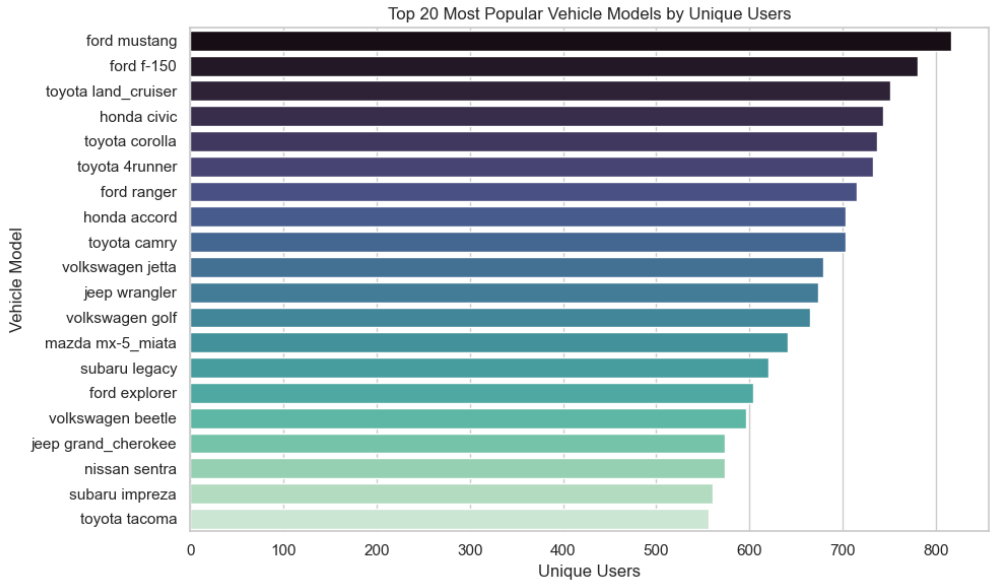



Figure 8: Top 20 vehicle models ranked by number of unique users.

The model-level ranking in Figure 8 provides a more detailed view of vehicle popularity than the make-level analysis. The top 20 models all have substantially more users than the remaining models outside the displayed ranking, although the difference between neighbouring entries within the top 20 is comparatively gradual

# 4 Fuel Usage

## 4.1 Outlier Removal

### 4.1.1 Top 5 currencies by transaction count

The five most common currencies by number of transactions are `$` (USD), `£` (GBP), `€` (EUR), `CA$` (CAD) and `R` (ZAR), as shown in Table 8. As anticipated in the assignment brief, the Rand is the fifth most common currency.

Table 8: Top 5 currencies by transaction count.

| Currency | Transactions |
|---|---|
| `$` (USD) | 741,889 |
| `£` (GBP) | 87,563 |
| `€` (EUR) | 59,258 |
| `CA$` (CAD) | 46,832 |
| `R` (ZAR) | 36,402 |

### 4.1.2 Per-currency outlier removal

The brief's own example, a South African user tagged with `$` reporting several hundred dollars for only a few tens of litres, is really a *cost-per-litre sanity check*: a fill-up's implied price (`total_spent`

/ litres) has to sit within the historically plausible retail fuel price band for that currency. Values wildly outside that band point either to a data-entry error (stray digits, a decimal point in the wrong place) or a currency tag that does not match the price actually paid.

For each of the top 5 currencies, a row is kept only if:

1. total_spent and litres are present and total_spent $> 0$;
2. litres falls in $[1, 300]$: below 1 L is not a real fill-up, and 300 L comfortably covers even large trucks and vans, so this only removes clear entry errors (e.g. an odometer reading captured in the gallons field);
3. the derived cost per litre falls inside a currency-specific plausible band, chosen generously to span 2005–2022 retail prices, including the 2021/22 price spike, while still excluding gross misclassifications (Table 9).

These bounds were checked against the empirical percentiles of each currency: in every case the 95th/99th percentile sits comfortably inside the band, while the 99.9th percentile jumps by one to two orders of magnitude: a clear break between genuine price variation and garbage data.

Table 9: Plausible cost-per-litre bounds used for outlier removal (local currency per litre).

| Currency | Band |
|---|---|
| USD ($) | 0.20 – 3.00 |
| GBP (£) | 0.50 – 3.00 |
| EUR (€) | 0.50 – 3.00 |
| CAD (CA$) | 0.50 – 2.50 |
| ZAR (R) | 6.00 – 24.00 |

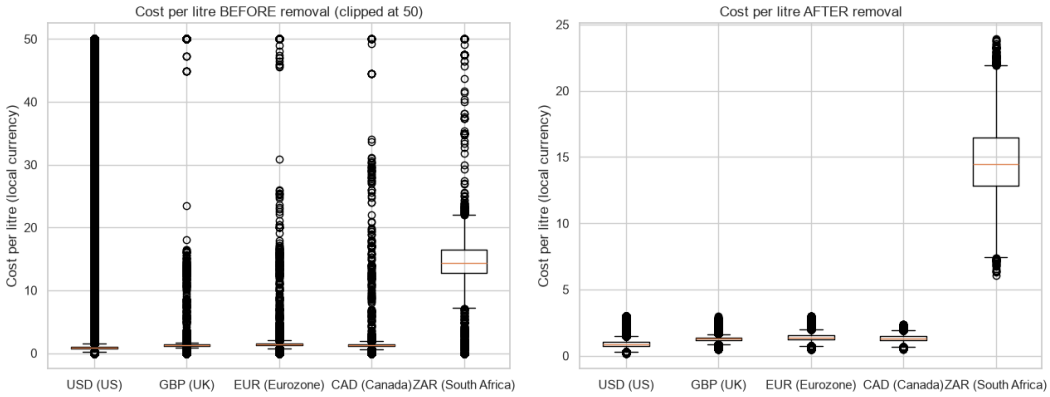



Figure 9: Cost per litre per currency before (left) and after (right) outlier removal. The pre-removal panel is clipped at 50 for legibility; the uncapped tail reaches several orders of magnitude higher.

### 4.1.3 Rows removed

Table 10 shows the number of rows removed per currency. In total, 55,761 of 971,944 rows (5.74%) across the top 5 currencies were removed as outliers, leaving 916,183 clean rows for the fuel efficiency analysis in Section 4. Removal rates range from 3.03% (CAD) to 6.32% (USD), low enough that the bounds are not simply discarding genuine variation, and consistent across currencies, which suggests the bands in Table 9 are reasonably well calibrated rather than arbitrarily strict for any one currency.

Table 10: Rows removed per currency.

| Currency | Before | Removed | % removed |
|---|---|---|---|
| USD (US) | 741,889 | 46,904 | 6.32 |
| GBP (UK) | 87,563 | 4,032 | 4.60 |
| EUR (Eurozone) | 59,258 | 2,201 | 3.71 |
| CAD (Canada) | 46,832 | 1,420 | 3.03 |
| ZAR (South Africa) | 36,402 | 1,204 | 3.31 |
| **Total** | **971,944** | **55,761** | **5.74** |

## 4.2 Fuel Efficiency

All analysis in this section uses the 916,183-row dataset remaining after the outlier removal described in Section 4.

### 4.2.1 Cost per litre per country, January 2022, converted to ZAR

Average January-2022 exchange rates to ZAR were computed from Nedbank's published daily rates for the 21 trading days in January 2022 (USD/ZAR, GBP/ZAR and EUR/ZAR); CAD/ZAR was derived as USD/ZAR ÷ USD/CAD, using a USD/CAD rate of 1.263 sourced from valutafx.com and ofx.com for the same month (Table 11).

Table 11: Exchange rates to ZAR used for Q1 (average, January 2022).

| Pair | Rate (1 unit = X ZAR) | Source |
|---|---|---|
| USD/ZAR | 15.53 | Nedbank daily rates, 3–31 Jan 2022 (21 trading days) |
| GBP/ZAR | 21.06 | Same source |
| EUR/ZAR | 17.59 | Same source |
| CAD/ZAR | 12.30 | Derived: USD/ZAR ÷ USD/CAD (USD/CAD ≈ 1.263) |
| ZAR/ZAR | 1.00 | Identity |

Table 12: Average cost per litre, January 2022, converted to ZAR.

| Country | $n$ | Local mean | ZAR per litre |
|---|---|---|---|
| United Kingdom (GBP) | 1,008 | 1.484 | 31.20 |
| Eurozone (EUR) | 1,165 | 1.589 | 27.95 |
| South Africa (ZAR) | 900 | 18.951 | 18.95 |
| Canada (CAD) | 1,658 | 1.474 | 18.13 |
| United States (USD) | 34,510 | 0.952 | 14.78 |

The UK and Eurozone show the highest ZAR-equivalent litre price, South Africa and Canada sit in the middle, and the US is the cheapest. This tracks fuel taxation rather than crude oil cost: US federal and state fuel tax is comparatively low, while UK and EU fuel duty and VAT are among the highest in the world. South Africa's price sits above the US/Canada level mainly because of the weaker Rand and the fuel levy and Road Accident Fund levy stacked on the base price, not because crude is more expensive there.

### 4.2.2 Detecting missed (unlogged) fill-ups

**Rule.** For a given user, fill-ups are sorted by date and the odometer gap between consecutive logged entries is computed. If every fill-up were logged, that gap should be reasonably close to the user's own *typical* refuelling distance. A gap much larger than that user's own median gap means at least one fill-up in between almost certainly went unlogged. Comparing each gap to the same user's own median, rather than a single global threshold, automatically accounts for the fact that different

people drive very different annual distances. Users need at least 3 valid gaps for their median to be meaningful. A gap $> 2\times$ that user's median gap is flagged as a likely missed fill-up.

Plotting the ratio of each gap to that user's own median gap gives a distribution heavily concentrated just above 1.0 (most fill-ups happen close to a driver's usual rhythm), with a long right tail past the 2.0x threshold rather than a second, separate peak, so the cutoff is a judgement call rather than a natural break in the data. Of 637,059 evaluated fill-up gaps, 113,403 (17.80%) exceed twice that user's median gap and are flagged as containing at least one missed fill-up. As a sensitivity check, 12.09% of gaps exceed a 2.5x threshold and 8.16% exceed 3.0x. Roughly 1 in 6 consecutive fill-up pairs shows a gap more than double that driver's usual refuelling distance, i.e. a fill-up almost certainly happened in between without being logged. This is a lower bound: a user who misses two fill-ups in a row, or who fills up partially every time, will not necessarily clear a 2x threshold.

### 4.2.3 Average distance per tank, per country

Table 13: Average distance per tank (km) by country.

| Country | Mean (km) | Median (km) |
|---|---|---|
| Eurozone (EUR) | 593.3 | 574.3 |
| United Kingdom (GBP) | 577.2 | 553.2 |
| South Africa (ZAR) | 541.2 | 523.0 |
| United States (USD) | 458.5 | 444.3 |
| Canada (CAD) | 441.6 | 426.8 |

The Eurozone has the largest average distance per tank (593 km). Eurozone and UK drivers lead, ahead of South Africa, with the US and Canada lowest. This lines up with vehicle mix rather than tank size: Europe and the UK have a much higher share of diesel vehicles, which are materially more fuel-efficient per litre than the petrol-dominant US/Canadian fleets, so a similarly-sized tank simply carries those vehicles further.

### 4.2.4 Do newer vehicles drive further between fill-ups?

Yes: distance per tank falls steadily and close to monotonically with vehicle age (correlation $= -0.176$), from approximately 510 km for cars 0–3 years old down to approximately 377 km for cars over 25 years old, across every age bracket tested in between. This is consistent with newer vehicles generally having better fuel efficiency and often larger tanks, and with older vehicles losing some efficiency (wear, outdated engine technology) over time.

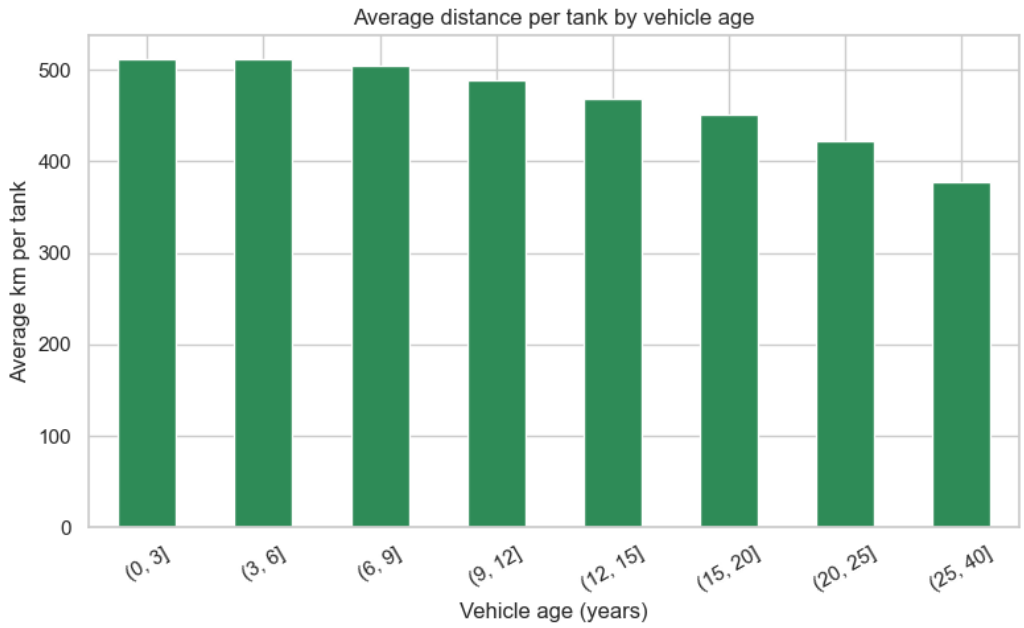



Figure 10: distance per tank by vehicle age

### 4.2.5 Fuel efficiency of top5 popular vehicles

The average fuel efficiency is calculated in Litres per 100km (L/100km). To ensure data integrity, entries with zero distance traveled or unrealistic efficiency values ($< 2$ or $> 50$ L/100km) are filtered out.

Table 14: Top 5 Most Popular Vehicles in South Africa by Unique Users

| Car Model | Avg Efficiency (L/100km) |
|---|---|
| Hilux | 12.1 |
| Pajero | 12.9 |
| Fortuner | 11.4 |
| Jimny | 9.2 |
| Polo | 7.8 |

The results indicate a preference for stronger utility vehicles (Hilux, Fortuner, Pajero) alongside smaller commuter cars (Polo, Jimny). The fuel efficiency values observed are realistic for these vehicle classes. For instance, the VW Polo, being a lightweight hatchback, exhibits higher efficiency (lower L/100km), while larger SUVs such as the Pajero and Hilux show higher consumption rates due to their weight and aerodynamic profile.

### 4.2.6 Vehicles with most fuel efficiency from each country

To determine the most fuel-efficient vehicle in each country, we calculated the mean L/100km for each model within each currency group. We applied realistic checks on the data by excluding outliers (efficiency $< 2$ or $> 30$ L/100km) to ensure the most efficient label was not assigned to erroneous data points.

Results show that the most efficient vehicles vary significantly by region, reflecting local market availability . In North America and Europe, hybrid and electric models (e.g., Prius Prime, electric drives) dominate the efficiency rankings, achieving values below 4 L/100km because they use less traditional fuel. In South Africa, the Renault Kwid, a small entry level hatchback, is the most efficient

Table 15: Most Fuel Efficient Vehicle Model per Country

| Country (Currency) | Most Efficient Model | Avg Efficiency (L/100km) |
|---|---|---|
| USA ($) | b_electric_drive | 2.58 |
| UK (£) | a250e | 2.86 |
| Eurozone (€) | 11 | 2.53 |
| Canada (CA$) | prius_prime | 3.58 |
| South Africa (R) | kwid | 5.23 |

among the top models in the country, with a realistic efficiency of $\approx 5.2$ L/100km. The kwid model available in South Africa is a traditional petrol car, which makes it have higher value in these results compared to the models from North America and European countries. These values align with real world manufacturer specifications, confirming the validity of our cleaned dataset.

### 4.2.7 Seasonal Fuel Efficiency in Canada

We analyzed the fuel efficiency of the top 5 Canadian vehicles across seasons to determine if weather impacts fuel consumption. We compared Winter (Dec, Jan, Feb) against Summer (Jun, Jul, Aug).

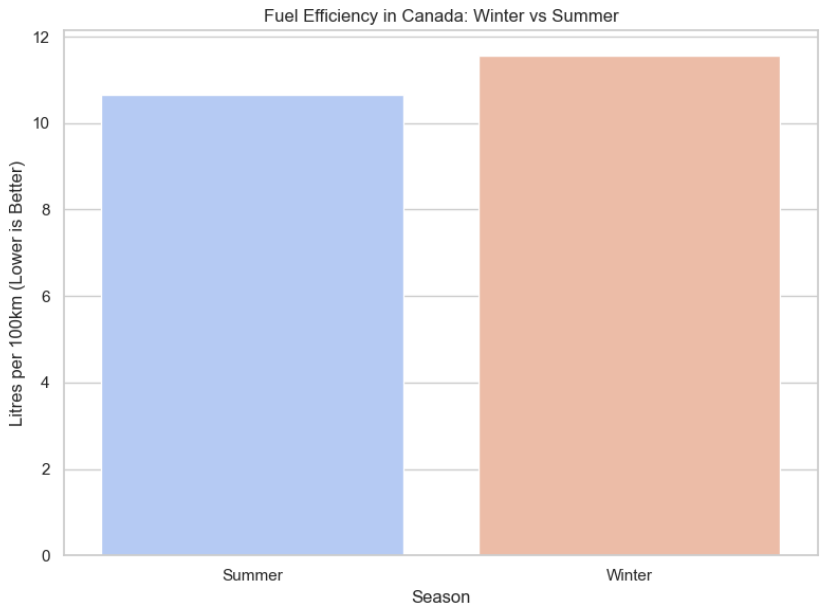



Figure 11: Average Fuel Efficiency (L/100km) for Top Canadian Vehicles: Winter vs. Summer.

As shown in Figure 11, there is a noticeable difference in fuel efficiency between seasons. Winter months shows higher L/100km values (lower efficiency) compared to Summer months. This is consistent with physical expectations: cold temperatures increase engine warm-up time and summer season makes warm engines run more efficiently. We compared Summer and Winter only because they have the most extreme temperatures.Focusing on these two extremes gives us a clear picture of seasonal changes, whereas Spring and Autumn represent transitional periods that are milder and make these changes harder to notice.

### 4.2.8: Correlation between fuel efficiency and other features

Fuel efficiency (`litres_per_100km`) is itself computed as `litres / killometre * 100`, so `litres`, `total_spent` and `cost_per_gallon` are excluded from this analysis: correlating a value against a term used to derive it is not a genuine relationship, it is circular. The real candidates are distance (`killometre`), vehicle age/year, and vehicle model/make.

`car_model` (1,945 levels) and `car_make` (29 levels) are not numeric, so each is converted to a numeric signal via 5-fold mean-target encoding: a row's encoded value is the average efficiency of *other* rows sharing its model or make, never its own. This keeps the categorical relationship usable without a row leaking into its own encoded value.

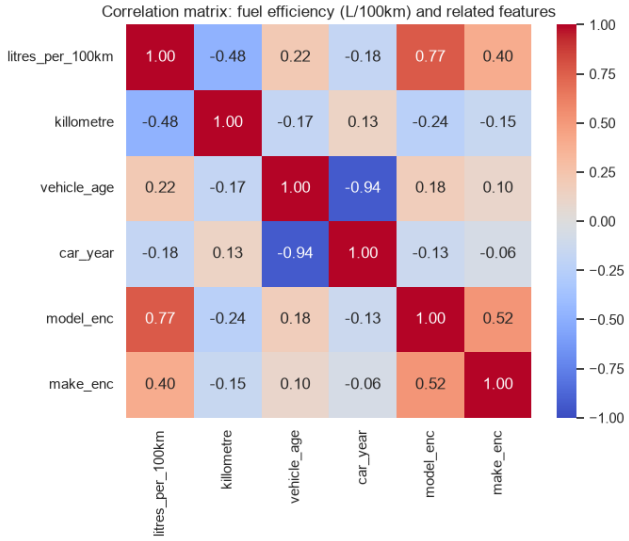



Figure 12: Correlation matrix: fuel efficiency (L/100km) and related features.

This confirms the expected pattern: model (0.77) is the strongest correlate by far, distance travelled is next ($-0.48$; more distance per tank implies a more efficient or larger-tanked car), make is moderate (0.40), and vehicle age is weaker (0.22). `car_year` and `vehicle_age` are almost perfectly anti-correlated ($-0.94$), since `vehicle_age = fueling_year - car_year` and `fueling_year` varies far less than `car_year`: the two columns carry almost the same information twice.

## 4.2.9: Random Forest feature importance

A Random Forest was trained on a 200,000-row sample using the same five numeric/encoded features as Q8, excluding `litres_per_100km`'s own algebraic inputs for the same reason as above (train $R^2 = 0.785$).

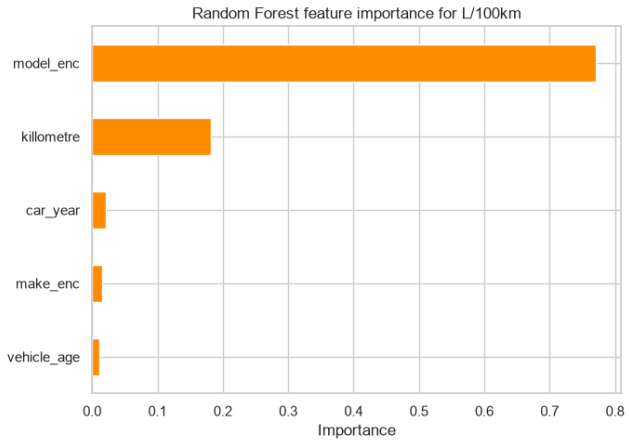



Figure 13: Random Forest feature importance for L/100km.

The Random Forest agrees with the correlation ranking on the top feature: model dominates (importance $\approx 0.77$), though it diverges further down. Distance keeps a clear second place (importance

$\approx 0.18$), much as its correlation magnitude suggested. `car_year`, `make_enc` and `vehicle_age` all drop to under 0.02 each, well below what their correlation coefficients ($-0.18$, $0.40$, $0.22$) would suggest on their own. Two things explain the gap. First, `car_year` and `vehicle_age` are almost perfectly collinear, so the forest can use either one and importance gets split between them rather than credited fully to one. Second, `make_enc` is itself largely explained by `model_enc` (0.52 correlation between the two, since make is a coarser version of model), so once the forest has split on model there is little independent signal left in make. Correlation looks at each feature in isolation; the Random Forest looks at marginal contribution once the other features are already in the tree, so redundant/collinear features naturally look far less important there even if their raw correlation is moderate.

## 4.3 Fuel Usage in SA

### 4.3.1 Focus on SA Drivers

The dataset was filtered to include only transactions from South Africa (Currency = 'R') and got 35,198 records.

### 4.3.2 Plot the fuel prices over time for SA

As shown in Figure 14, the fuel price trends over time and the distribution of refueling events by day of the week.Over time, fuel prices have increased significantly, rising from roughly R7.00 per litre around 2008 to over R21.00 per litre by 2022. Despite the general increase, the prices are highly volatile, showing frequent short-term fluctuations with sharp spikes and sudden drops over the years.

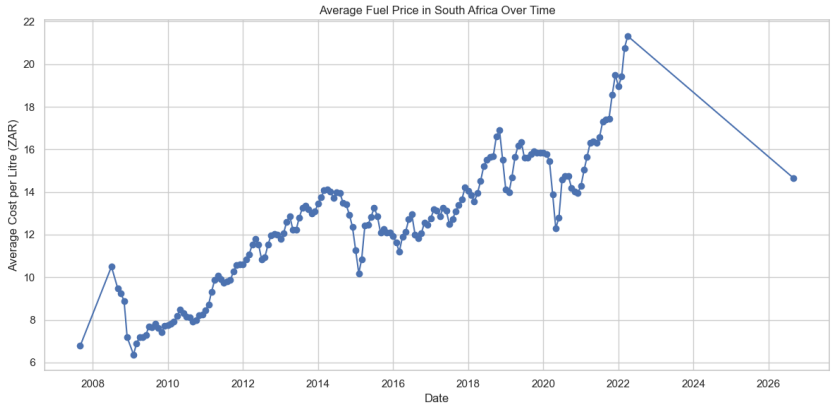



Figure 14: Fuel Price Trends in South Africa Over Time.

### 4.3.3 Difference in the number of people refueling on a Tuesday vs other days

As shown in Figure 15, Tuesday has the highest number of refuels compared to any other day of the week. This spike occurs because South African fuel prices are officially adjusted on the first Wednesday of every month, with the new prices taking effect at midnight on Tuesday. Consequently, motorists actively alter their driving routines, rushing to fill up their vehicles on Tuesdays to avoid impending price increases and save money before the new rates apply.

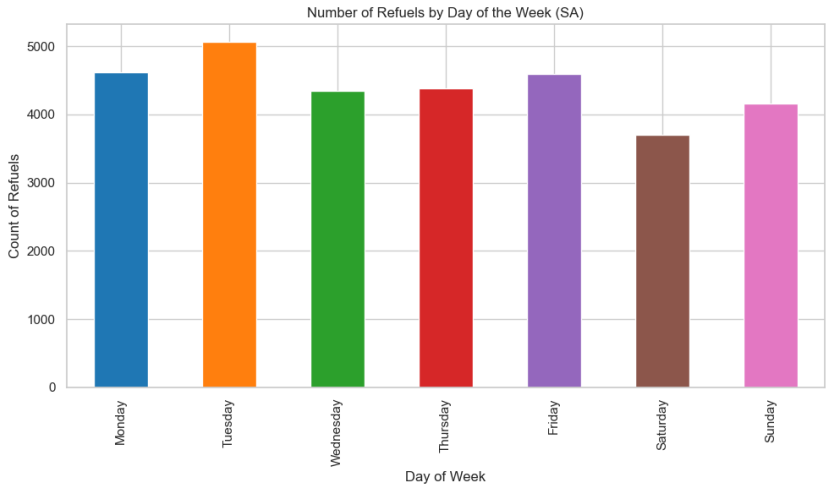



Figure 15: Fuel Price Trends in South Africa Over Time.

### 4.3.4 Reduce dataset to only the entries on the 1st Tuesday and 1st Wednesday in SA every month

To isolate the relevant days for analysis, the dataset was filtered to include only entries occurring on the first Tuesday and first Wednesday of each month. This was achieved by extracting the day of the week and the date of the month for each logbook entry. Since the first occurrence of any weekday always falls within the first seven days of a month, the data was restricted to dates where the day of the month was less than or equal to 7, and the weekday corresponded strictly to Tuesday or Wednesday.

### 4.3.5 Indicating Price Increase or Decrease

Table 16: Refueling Counts on First Tuesday/Wednesday by Price Direction

| Day | Price Direction | Count of Refuels |
|---|---|---|
| First Tuesday | Price Down | 323 |
| First Tuesday | Price Up | 1346 |
| First Wednesday | Price Down | 424 |
| First Wednesday | Price Up | 496 |

The results in Table 16 show how drivers respond to monthly fuel price adjustments in South Africa.

- First Tuesday (Price Up - 1,346 refuels): There is a massive spike in refuels. Motorists rush to fuel stations on Tuesday afternoon/evening to lock in the lower price before the midnight price hike.
- First Tuesday (Price Down - 323 refuels): Refuel numbers drop dramatically. Motorists intentionally postpone refueling on Tuesday because they know petrol will be cheaper starting at midnight.
- First Wednesday (Price Going UP - 496 refuels): refueling counts drop sharply compared to Tuesday (down from 1,346 to 496) because most drivers already filled their tanks the day before.
- First Wednesday ( Price going Down - 424 refuels): Counts increase compared to Tuesday (up from 323 to 424) as drivers who postponed filling up finally head to the pumps to take advantage of the newly reduced rate.

### 4.3.6 Do more people refuel on the first Wednesday of the month when the prices goes down?

**No**, fewer people refuel on the first Wednesday of the month when prices go down compared to when they go up. According to the data, there are **424 refuels** on the first Wednesday when prices decrease, which is less than the **496 refuels** recorded on Wednesdays when prices increase.

### 4.3.7 Do more people refuel on the first Tuesday of the month when the prices goes up?

**Yes**, significantly more people refuel on the First Tuesday of the month when fuel prices go up. According to the dataset, there are **1,346 refuels** recorded on the First Tuesday prior to a price increase, which is substantially higher than the **323 refuels** recorded when a price decrease is expected.

# References

[1] Jupyter notebook, *DataExploration.ipynb*, “Feature Engineering and Vehicle Exploration”, September 2026.